



PROJECT PROPOSAL / SEPTEMBER 2026

Adaptive Musical Practice System

for monophonic instruments

A web app that listens, learns, and turns recurring pitch and timing problems into focused practice.

The idea in one line Do more than flag a wrong note: understand when it keeps happening, why the conditions matter, and what the player should practise next.

At a glance

The project

This project explores a simple question: *what if a practice app remembered how you play?* The proposed system listens to a musician performing a one-note-at-a-time exercise, compares the performance with a reference, and gives quick feedback on pitch and timing. More importantly, it keeps the useful results from each session. Over time, those results reveal habits that a single tuner reading cannot—for example, notes becoming flat in the upper register or timing becoming rushed above a certain tempo.

Listen

Capture a monophonic performance through the browser microphone.

Compare

Align played notes with a known exercise and calculate pitch and timing error.

Learn

Find repeated, tempo-dependent, register-dependent, and section-specific patterns.

Adapt

Recommend a focused task, then measure whether it actually helped.

First-release boundaries

The first version supports one primary monophonic instrument and one note at a time. It does not attempt chord recognition, physical-technique diagnosis, custom neural-network training, or a mobile app. Keeping these boundaries clear makes an end-to-end, testable product realistic within eight weeks.

Contents

At a glance	1
1 Why this project?	3
1.1 The problem in practical terms	3
2 Aim and project objectives	3
3 What the experience looks like	4
3.1 Live practice	4
3.2 Session review	4
3.3 Practice history	4
3.4 Adaptive follow-up	4
4 How the system works	5

4.1	1. Audio capture and note detection	5
4.2	2. Reference alignment and comparison	5
4.3	3. Performance profile	5
4.4	4. Recommendation and readable report	5
5	Features for the first release	6
6	Technical approach	6
6.1	Proposed stack	6
6.2	Data model	7
7	Scope: what fits and what waits	7
8	Eight-week build plan	8
9	Evaluation plan	8
9.1	Audio and alignment	9
9.2	System and data pipeline	9
9.3	Pattern detection and adaptation	9
9.4	AI report grounding	9
10	Risks and practical responses	10
11	Expected outcome	10
12	Where it could go next	10
13	Conclusion	11

1 Why this project?

Learning an instrument is built on a familiar loop: play, notice, adjust, repeat. A tuner can show that the current note is flat. A metronome can reveal that the beat is drifting. A teacher can point out a difficult bar. These are all valuable, but they rarely create a detailed memory of what happens across many independent practice sessions.

That missing history matters. A player may sound accurate at 70 BPM but lose control at 90 BPM. They may consistently rush after rests, struggle only in a high register, or stumble over the same two-note transition. Any one attempt can look like an isolated mistake; several sessions can reveal a pattern.

The gap

Most practice tools answer, “What went wrong just now?” This project also asks, “What keeps going wrong, under which conditions, and what should happen next?”

The result will be a web application that connects immediate feedback with longer-term, personalised practice. It is designed as a supportive practice companion rather than a replacement for a teacher or a musician’s own judgment.

1.1 The problem in practical terms

Generic feedback such as “play in tune,” “slow down,” or “watch the rhythm” is hard to act on. Useful feedback should be specific enough to guide the next five minutes of practice. The system therefore needs to distinguish between:

- a one-off error and a recurring habit;
- a general weakness and a problem tied to a note range, section, or tempo;
- normal variation and a meaningful improvement or regression;
- detecting a problem and choosing a sensible response to it.

2 Aim and project objectives

The aim is to build a web-based practice system that listens to monophonic playing, explains performance clearly, and uses session history to make the next practice task more useful.

The project has five connected objectives:

1. **Hear the performance.** Detect pitch, note onset, duration, and basic level information from microphone audio.
2. **Compare it fairly.** Align the performance with reference notes even when the player speeds up or slows down slightly.
3. **Explain the session.** Calculate note-level and section-level pitch, timing, and stability measures.
4. **Build a longer-term picture.** Combine sessions to find repeated or condition-based problems and track change over time.
5. **Close the practice loop.** Suggest a targeted exercise, record the follow-up result, and report whether the intervention helped.

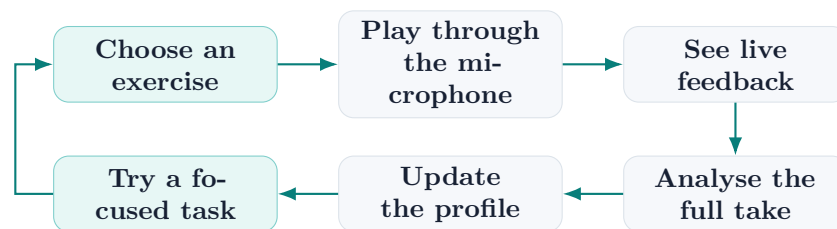
Example feedback

Expected: G4 **Played:** G4 **Pitch:** −8 cents **Timing:** 34 ms late

This immediate measurement can later become a richer observation: “The F4–G4 transition is still the main issue, especially above 80 BPM. Start at 72 BPM and increase only after three stable repetitions.”

3 What the experience looks like

The core interaction should feel like a natural practice session, not an engineering dashboard. A player chooses an exercise, checks the tempo, starts the microphone, and plays. During the attempt, the interface shows only lightweight feedback. Once the session ends, the app performs a deeper analysis and updates the player’s history.



3.1 Live practice

While the player performs, the browser displays the current detected note, its distance from the target in cents, and simple early/late timing feedback. A metronome and restrained visual cues keep the screen useful without distracting from the instrument.

3.2 Session review

After the take, the player sees overall accuracy, average error, note stability, difficult sections, and note-level results. Graphs show where pitch or timing changed across the exercise and how the take compares with previous attempts.

3.3 Practice history

The history view highlights patterns rather than just listing scores. Examples include “timing accuracy drops above 85 BPM,” “the upper register is improving,” and “bar 6 has been difficult in four of the last five attempts.”

3.4 Adaptive follow-up

Each recognised problem maps to a practical intervention. If timing becomes unstable above 85 BPM, the app might isolate the affected section, begin at 72 BPM, request five repetitions, and then retry at 80 BPM. The following result is linked to that intervention so its effect can be measured.

4 How the system works

The system is divided into four layers. Each layer produces structured data for the next, keeping the audio measurements, recommendations, and language-model output clearly separated.

4.1 1. Audio capture and note detection

The browser captures microphone audio with the Web Audio API and processes short frames in an `AudioWorklet`. A lightweight method such as YIN provides low-latency pitch estimates for live feedback. The pipeline also detects note onset, duration, and signal level.

Once a session ends, a more detailed pass can use signal-processing tools or an existing pretrained model such as CREPE or Basic Pitch. The choice will be based on early tests for the selected instrument, accuracy, latency, and deployment cost. No new neural network will be trained.

4.2 2. Reference alignment and comparison

Each exercise stores its expected note sequence, durations, sections, and tempo. Dynamic Time Warping (DTW) aligns the played sequence with the reference when timing is not perfectly uniform. For each matched note, the system calculates:

- pitch deviation in cents and whether it is sharp or flat;
- onset difference and relative duration;
- pitch stability during the sustained portion;
- confidence and signal-quality information;
- section, register, tempo, and transition context.

4.3 3. Performance profile

Session metrics are stored in PostgreSQL and aggregated into a personal performance profile. Statistical summaries and transparent rules identify repeated pitch or timing errors, difficult note ranges and transitions, tempo thresholds, section-specific issues, and trends across time. The profile also records previous interventions and their outcomes.

4.4 4. Recommendation and readable report

A rule-based recommendation layer connects recognised patterns to a small library of appropriate exercises. An existing hosted language model then converts the validated results into readable feedback. It receives structured facts—not raw audio—and must stay within those facts. The system validates important statements against the source metrics before displaying them.

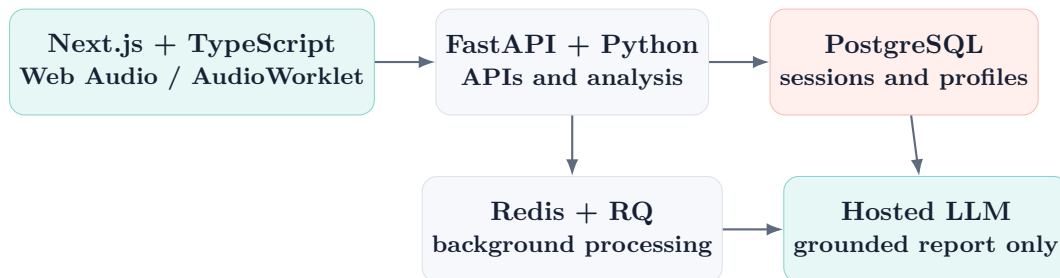
A safe role for the language model

The language model is a writing layer, not the measurement layer. It can make a report easier to read, but it cannot invent a weakness, diagnose technique, or claim improvement that is absent from the calculated data.

5 Features for the first release

Area	What the player can do
User setup	Register, sign in, create a profile, and select the supported instrument.
Exercise library	Browse monophonic exercises by tempo and difficulty; load reference notes and sections.
Live practice	Use microphone input, metronome, current-pitch display, cents deviation, and basic timing cues.
Session analysis	Review pitch and timing accuracy, note stability, note-level results, difficult sections, and graphs.
History	Revisit sessions, view progress, and see repeated, improving, tempo-dependent, or register-dependent problems.
Adaptive practice	Receive section isolation, repetition, and tempo-step tasks connected to the detected problem.
Practice report	Read a grounded summary of progress, main issues, and the suggested focus for the next session.

6 Technical approach

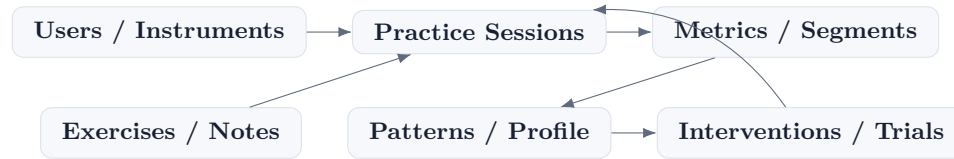


6.1 Proposed stack

Frontend	Next.js, TypeScript, HTML/CSS, Web Audio API, and AudioWorklet.
Backend	Python, FastAPI, SQLAlchemy, and PostgreSQL.
Audio analysis	NumPy, SciPy, librosa, DTW, YIN, and—if testing supports it—CREPE or Basic Pitch.
Background work	Redis with RQ or a similar Python task queue.
AI report	An existing hosted language model supplied with validated, structured session facts.
Deployment	Cloud-hosted frontend and API, managed PostgreSQL, and optional object storage for consented recordings.

6.2 Data model

The database must preserve both individual evidence and the longer-term interpretation built from it. The main relationships are:



Raw recordings are optional rather than required. If enabled, they should use explicit consent, clear retention rules, and object storage rather than the relational database.

7 Scope: what fits and what waits

Included in version one

- One primary monophonic instrument
- One note at a time
- Pitch and timing analysis
- Reference-note comparison
- Practice history
- Repeated-error detection
- Personal performance profile
- Adaptive practice suggestions
- Grounded AI-written reports
- Architecture that can later support other monophonic instruments

Saved for later

- Polyphonic or chord detection
- Piano and guitar chord analysis
- Real-time online duets
- Custom neural-network training
- Medical or physical-technique diagnosis
- A large social network
- Native mobile applications
- Automatic claims about musical expression without measurable evidence

The scope is intentionally narrow. Reliable note alignment, meaningful history, and a working recommendation loop are more valuable for this project than broad but shallow instrument support.

8 Eight-week build plan

Week	Focus	Key work	End-of-week result
1	Audio foundations	Confirm the primary instrument; set up the project; test microphone capture; compare pitch methods; measure basic accuracy and delay; draft the schema.	Working audio prototype
2	Live practice	Add AudioWorklet processing, pitch and onset detection, a metronome, and the first live-feedback interface.	Play and receive immediate feedback
3	Reference comparison	Build exercises and reference notes; implement DTW alignment; calculate pitch and timing error; create the first session review.	Full take can be analysed
4	Backend and storage	Add authentication, exercise and session APIs, PostgreSQL persistence, and background jobs.	Complete history can be saved and loaded
5	Performance profile	Detect repeated, tempo-dependent, register-dependent, and section-specific patterns; track improvement and regression.	Common habits can be described
6	Adaptive practice	Create the intervention library; connect patterns to tasks; track trials and update recommendations from outcomes.	Practice changes based on history
7	Report and interface	Integrate the hosted language model; constrain and validate reports; build dashboard and progress pages; refine the experience.	Complete user-facing flow
8	Test and deliver	Test audio conditions, APIs, database, latency, alignment, recommendations, and report grounding; fix issues; deploy; prepare documentation.	Final working system

Suggested decision gate after Week 1

Continue with the chosen instrument and detector only if the prototype reaches usable accuracy and latency in ordinary practice conditions. If it does not, reduce the supported range or simplify the first-release exercise set before building the remaining layers.

9 Evaluation plan

Evaluation will test the chain from audio input to practice recommendation, not just whether the interface works.

9.1 Audio and alignment

- Pitch-detection accuracy against labelled or synthesised notes
- Note/onset detection accuracy and onset error in milliseconds
- Alignment quality for performances with controlled timing variation
- End-to-end processing latency during live feedback
- Robustness across reasonable microphone distance and background noise

9.2 System and data pipeline

- API response time and background-analysis duration
- Reliable processing and storage of complete sessions
- Correct links between session evidence, profile patterns, and interventions

9.3 Pattern detection and adaptation

Using controlled test histories, the team will check whether the system correctly identifies repeated errors, note-range and tempo conditions, and improvement or regression. Recommendations will be reviewed to ensure that they match the detected problem. Intervention trials must preserve both the assigned task and the measured follow-up result.

9.4 AI report grounding

Each generated statement will be checked against the structured input. Reports pass only when stated problems, comparisons, and recommendations are supported by the analysis. Unsupported claims should be rejected or replaced with deterministic template text.

10 Risks and practical responses

Risk	Planned response
Noisy or low-quality input	Show input-level guidance, store confidence values, ignore unreliable frames, and test with realistic devices.
Pitch errors for a specific instrument	Select one instrument early, benchmark several methods, and constrain the supported register if needed.
DTW produces musically odd matches	Use note and section constraints, inspect difficult cases, and surface low-confidence alignments.
Too little history for personal patterns	Require minimum evidence, label early observations as tentative, and use session-level feedback first.
Recommendations feel repetitive	Build a small but varied intervention library and use previous trial outcomes when ranking tasks.
The language model overstates results	Supply validated facts only, constrain output structure, verify claims, and fall back to templates.
Eight-week scope expands	Treat monophony, one instrument, and the end-to-end practice loop as non-negotiable boundaries.

11 Expected outcome

At the end of eight weeks, a musician should be able to:

1. choose a monophonic exercise and practise through a microphone;
2. receive useful pitch and timing feedback;
3. inspect a detailed, note-level session review;
4. see changes across several practice sessions;
5. recognise recurring problems and the conditions that trigger them;
6. receive a focused practice task linked to the evidence;
7. repeat the task and see whether the intervention helped; and
8. read a concise report that accurately reflects the calculated data.

The finished prototype will demonstrate a complete integration of browser audio, backend services, signal processing, persistent data, longitudinal analysis, adaptive recommendations, and a carefully limited use of generative AI.

12 Where it could go next

Once the core loop is reliable, the system could grow to support more monophonic instruments, phrasing and expression measures, teacher/student accounts, shareable progress reports, richer exercise generation, mobile access, and eventually carefully scoped polyphonic analysis. Those ideas are valuable, but they should follow evidence that the first version produces accurate measurements and genuinely useful practice decisions.

13 Conclusion

This is not simply a pitch detector with a dashboard. Its central contribution is the connection between today's note, last week's pattern, and the next useful practice action.

By using existing audio and language models where appropriate, the team can focus on the harder product question: how to turn performance data into feedback that remains specific, honest, and helpful over time. A narrow monophonic scope makes that goal realistic within eight weeks while leaving a clear path for future instruments and richer musical analysis.



Listen. Learn. Practise with purpose.